SE601P

(Software construction and development Practical)

Objectives + Subjective from Week 01 – 16

Prepared By: **ALI ABBAS**

BS Software Engineering

YouTube channel: **VU Facts**

https://www.youtube.com/@vufacts

VU Facts

☑ WEEK 1 — Object Oriented Software Development

◈ A. MCQs (Directly from Handouts)

1. Object-oriented software development is based on the concept of:

- a) Functions
- b) Objects ✔
- c) Procedures
- d) Modules

2. An object is:

- a) A function
- b) A variable
- c) An instance of a class ✔
- d) A method

3. Which OOP principle allows code reuse?

- a) Encapsulation
- b) Abstraction
- c) Inheritance ✔
- d) Polymorphism

4. Which concept hides internal implementation details?

- a) Inheritance
- b) Polymorphism
- c) Information Hiding ✔
- d) Association

5. Object lifetime refers to:

- a) Object size
- b) Object execution
- c) Time an object exists in memory ✔
- d) Object name

◈ B. Conceptual MCQs

VU Facts

1. Encapsulation improves software quality mainly by:

- a) Increasing execution speed
- b) Reducing complexity ✔
- c) Increasing memory usage
- d) Removing classes

2. Which relationship shows a strong “part-of” dependency?

- a) Association
- b) Aggregation
- c) Composition ✔
- d) Inheritance

◈ C. Long Questions (Lab-Based)

Q1. Explain Object-Oriented Software Development and its main principles.

Answer:
Object-oriented software development is a programming methodology that focuses on creating software using objects, which are instances of classes. A class defines both data and behavior, making software modular and reusable.

The main principles include encapsulation, which bundles data and methods into a single unit and restricts direct access. Inheritance allows one class to reuse properties and behaviors of another class. Polymorphism enables objects to behave differently in different contexts. Abstraction simplifies complex systems by exposing only essential features through interfaces or abstract classes. These principles improve maintainability, flexibility, and scalability.

Q2. Differentiate between association, aggregation, and composition.

Answer:
Association represents a general relationship between objects where they are connected but independent. Aggregation is a weak “has-a” relationship where objects can exist independently of the main object. Composition is a strong “has-a” relationship where the composed objects cannot exist without the main object. Composition indicates ownership and lifecycle dependency, while aggregation does not.

VU Facts

◈ D. Conceptual Long Question

Q3. Explain information hiding and its importance in software design.

Answer:
Information hiding is a design principle that restricts access to internal details of a class or module and exposes only necessary interfaces. It reduces dependency between components, improves security, and enhances maintainability. Changes in internal implementation do not affect other parts of the system, making software easier to modify and extend.

☑ WEEK 2 — Object Relationships & Polymorphism

◈ A. MCQs (From Lab Content)

1. Objects share information using:

- a) Compilation
- b) Method calls ✔
- c) Debugging
- d) Linking

2. Passing object data to a method is called:

- a) Message passing
- b) Serialization
- c) Parameter passing ✔
- d) Inheritance

3. A superclass and subclass relationship represents:

- a) Aggregation
- b) Composition
- c) Generalization ✔
- d) Association

4. Overriding allows a subclass to:

- a) Remove a method
- b) Redefine inherited behavior ✔
- c) Create objects

VU Facts

- d) Hide methods

◈ B. Conceptual MCQs

1. Interfaces mainly support:

- a) Encapsulation
- b) Polymorphism ✔
- c) Aggregation
- d) Serialization

2. Method overloading is an example of:

- a) Runtime polymorphism
- b) Compile-time polymorphism ✔
- c) Inheritance
- d) Abstraction

◈ C. Long Questions

Q1. Explain inheritance, extension, and overriding with an example.

Answer:
Inheritance allows a subclass to reuse properties and methods of a superclass, forming a hierarchical relationship. Extension enables the subclass to add new methods or properties not present in the superclass. Overriding allows the subclass to redefine an inherited method to provide specific behavior. For example, a `Dog` class can inherit from an `Animal` class, add a `bark()` method, and override the `move()` method.

Q2. Explain polymorphism with a suitable example.

Answer:
Polymorphism allows objects to take multiple forms and respond differently to the same method call. It is achieved through method overloading, overriding, or interfaces. For example, a `Vehicle` reference can point to `Car` or `Plane` objects, and calling the `move()` method results in different behaviors based on the object type. This improves flexibility and extensibility.

VU Facts

◈ D. Conceptual Question

Q3. What is realization and how is it different from inheritance?

Answer:
Realization occurs when a class implements an interface and provides definitions for its methods. Inheritance shares implementation and behavior from a superclass, whereas realization shares behavior definition without implementation. Realization promotes loose coupling and flexibility.

☑ WEEK 3 — Generics, Distributed Objects & Serialization

◈ A. MCQs (Lab-Based)

1. Generic programming is used to:

- a) Improve UI
- b) Write reusable code ✔
- c) Increase memory usage
- d) Avoid classes

2. Generics use:

- a) Objects
- b) Methods
- c) Type parameters ✔
- d) Interfaces

3. Distributed objects communicate using:

- a) Local calls
- b) RPC or messaging protocols ✔
- c) Constructors
- d) Loops

4. Serialization converts an object into:

- a) Method
- b) Class
- c) Byte stream ✔
- d) Interface

VU Facts

◈ B. Conceptual MCQs

1. Deserialization means:

- a) Object deletion
- b) Byte stream to object ✔
- c) Object cloning
- d) Memory allocation

2. Distributed objects mainly support:

- a) Centralized systems
- b) Scalability ✔
- c) Local execution
- d) Static binding

◈ C. Long Questions

Q1. Explain generic programming and its benefits.

Answer:
Generic programming allows writing reusable code that works with different data types using type parameters. It reduces code duplication and improves flexibility. Languages like Java and C++ use generics to create container classes such as lists and maps. Generic programming improves maintainability and ensures type safety.

Q2. Explain object serialization and its importance.

Answer:
Object serialization is the process of converting an object's state into a byte stream so it can be stored or transmitted. Deserialization reconstructs the object from the byte stream. Serialization is essential in distributed systems, persistence, caching, and messaging, as it preserves object state across different systems.

◈ D. Conceptual Long Question

VU Facts

Q3. Explain the role of distributed objects in modern software systems.

Answer:
Distributed objects allow software components to run on different systems and communicate through protocols such as HTTP or RPC. They enable scalable, modular, and fault-tolerant systems by distributing workload across multiple machines. Technologies like Java RMI and web services implement distributed objects.

☑ WEEK 6 — CODE REVIEW & SOFTWARE TOOLS

◈ A. MCQs (From Lab / Handouts)

1. Code review is mainly performed to:

- a) Increase execution speed
- b) Identify issues and improve code quality ✔
- c) Deploy software
- d) Compile code

2. Which tool provides pull request–based code review?

- a) Jira
- b) GitHub ✔
- c) Selenium
- d) JMeter

3. CodeClimate is mainly used for:

- a) Bug tracking
- b) Code analysis and detecting code smells ✔
- c) Documentation
- d) Testing

4. Bug tracking tools are used to:

- a) Write code
- b) Track and manage defects ✔
- c) Compile software
- d) Debug applications

VU Facts

◈ B. Conceptual MCQs

1. Code review helps reduce:

- a) Software documentation
- b) Technical debt ✔
- c) Version control
- d) User requirements

2. Bug tracking tools are most important for:

- a) Small scripts
- b) Large development teams ✔
- c) Single-user applications
- d) Static websites

◈ C. Long Questions (Lab-Based)

Q1. Explain code review and its importance in software development.

Answer:
Code review is a critical activity in software development where developers examine each other's code to identify errors, suggest improvements, and ensure adherence to coding standards. It improves code quality by detecting bugs early, reducing technical debt, and ensuring maintainability and scalability. Code review also promotes knowledge sharing and collaboration among team members, resulting in more reliable and consistent software.

Q2. Explain bug tracking tools and their role in software projects.

Answer:
Bug tracking tools are used to record, manage, and monitor bugs and issues in software applications through a centralized system. These tools allow teams to report bugs, assign issues, set priorities, and track progress until resolution. By integrating with development tools, bug tracking systems help teams fix issues early, reduce production failures, and improve overall software stability. Examples include Jira, Bugzilla, Mantis, and Redmine.

◈ D. Conceptual Long Question

VU Facts

Q3. Differentiate between code review tools and bug tracking tools.

Answer:
Code review tools focus on analyzing source code to identify quality issues, enforce standards, and suggest improvements before code is merged. Bug tracking tools, on the other hand, manage reported defects after code execution or testing. While code review tools prevent defects, bug tracking tools help manage and resolve them systematically during development.

☑ WEEK 7 — JIRA (PROJECT & ISSUE MANAGEMENT)

◈ A. MCQs (Lab-Based)

1. Jira is mainly used for:

- a) Code compilation
- b) Issue and project tracking ✔
- c) Unit testing
- d) Debugging

2. An Epic in Jira is used to:

- a) Fix bugs
- b) Represent a large body of work ✔
- c) Assign users
- d) Deploy software

3. Child issues in Jira are created to:

- a) Delete tasks
- b) Break down work ✔
- c) Compile code
- d) Review code

◈ B. Conceptual MCQs

1. Jira improves project management by:

- a) Writing code

VU Facts

- b) Tracking progress ✔
- c) Compiling software
- d) Debugging errors

◈ C. Long Questions (Lab-Based)

Q1. Explain the steps to create and manage a project in Jira.

Answer:
To manage a project in Jira, first create an account and then create a new project by selecting a suitable template. After setting up the project, issues such as tasks or bugs are added. Epics are created in the roadmap to represent major work items, and child issues are linked to epics. Timelines are set to track progress, and the roadmap view is used to monitor overall project status effectively.

◈ D. Conceptual Question

Q2. How does Jira help in monitoring project progress?

Answer:
Jira provides roadmap views, issue tracking, and timelines that allow project managers to monitor task completion and project status. These features help identify delays, manage workloads, and ensure timely delivery of software projects.

☑ WEEK 8 — ASANA (TASK MANAGEMENT)

◈ A. MCQs (Lab-Based)

1. Asana is mainly used for:

- a) Testing
- b) Task and project management ✔
- c) Code review
- d) Debugging

2. Timeline in Asana is used to:

VU Facts

- a) Write code
- b) Set task deadlines ✔
- c) Compile programs
- d) Debug errors

◈ B. Conceptual MCQs

1. Project manager dashboard in Asana helps to:

- a) Write code
- b) Monitor task progress ✔
- c) Run tests
- d) Push code

◈ C. Long Questions

Q1. Explain how tasks are created and managed in Asana.

Answer:
In Asana, a new project is created first, after which tasks are added and assigned to team members. Each task is given a timeline to define start and end dates. The project manager can monitor progress through the dashboard, while developers can view pending and completed tasks. This structure improves collaboration and ensures timely task completion.

☑ WEEK 9 — GOOD & BAD CLASS DESIGN

◈ A. MCQs (Lab-Based)

1. Encapsulation means:

- a) Making variables public
- b) Hiding internal state ✔
- c) Removing constructors
- d) Avoiding methods

2. Single Responsibility Principle states that:

VU Facts

- a) A class should have many responsibilities
- b) A class should have one responsibility ✔
- c) A class should avoid methods
- d) A class should expose variables

◈ B. Conceptual MCQs

1. Poor class design leads to:

- a) Better maintainability
- b) Tight coupling ✔
- c) Clear responsibilities
- d) Better debugging

◈ C. Long Questions

Q1. Explain characteristics of a well-designed class.

Answer:
A well-designed class follows encapsulation by keeping data private and providing controlled access through methods. It follows the single responsibility principle by focusing on one clear purpose. Proper constructors ensure valid object initialization, while exception handling improves reliability. Such design enhances maintainability, flexibility, and readability of the code.

Q2. How can a bad class design be improved?

Answer:
Bad class design can be improved by making member variables private, adding proper constructors, handling exceptions, and following design principles such as encapsulation and single responsibility. These improvements reduce errors and make the class more maintainable and extensible.

✅ WEEK 10 — MISRA C++ CODING STANDARD

VU Facts

◈ A. MCQs (Lab-Based)

1. MISRA C++ is used to write:

- a) Fast code
- b) Safe and reliable code ✔
- c) Short programs
- d) Dynamic code

2. MISRA recommends avoiding:

- a) Standard libraries
- b) Dynamic memory allocation ✔
- c) Loops
- d) Classes

◈ B. Conceptual MCQs

1. The main goal of MISRA C++ is to:

- a) Increase execution speed
- b) Improve safety and maintainability ✔
- c) Reduce file size
- d) Improve UI

◈ C. Long Questions

Q1. Explain the purpose of MISRA C++ coding standards.

Answer:
MISRA C++ coding standards provide guidelines for writing safe, reliable, and maintainable C++ code. These rules help developers avoid undefined behavior, reduce runtime errors, and ensure consistent coding practices. MISRA standards are especially important in safety-critical systems where reliability is essential.

Q2. Explain any two MISRA C++ rules with examples.

VU Facts

Answer:
MISRA recommends maintaining a consistent coding style to improve readability and reduce errors. It also advises avoiding dynamic memory allocation because it can lead to memory leaks and unpredictable behavior. Using static or stack-based memory improves safety and reliability.

☑ WEEK 11 — DEVOPS

MCQs (Lab / Fact-Based)

1. DevOps mainly focuses on improving collaboration between:

- a) Developers and testers
- b) Developers and operations ✔
- c) Testers and managers
- d) Users and developers

2. Which tool is commonly used for Continuous Integration?

- a) Docker
- b) Kubernetes
- c) Jenkins ✔
- d) Ansible

3. Git is an example of:

- a) Programming language
- b) CI tool
- c) Version Control System ✔
- d) Monitoring tool

4. Which DevOps practice ensures frequent code integration?

- a) Continuous Delivery
- b) Continuous Integration ✔
- c) Virtualization
- d) Containerization

5. Docker is mainly used for:

- a) Configuration management
- b) Monitoring
- c) Containerization ✔
- d) Version control

VU Facts

6. Kubernetes is used for:

- a) Version control
- b) Container orchestration ✔
- c) CI automation
- d) Monitoring

MCQs (Conceptual)

1. The primary goal of DevOps is to:

- a) Increase documentation
- b) Improve collaboration and automation ✔
- c) Separate teams
- d) Elik98minate testing

2. CI/CD mainly helps in:

- a) Delaying releases
- b) Automating integration and delivery ✔
- c) Increasing manual work
- d) Avoiding testing

3. Continuous Integration helps in:

- a) Late error detection
- b) Early error detection ✔
- c) Manual testing
- d) Skipping builds

4. DevOps practices improve:

- a) Only coding
- b) Only testing
- c) Software delivery speed ✔
- d) Hardware usage

5. Continuous Delivery ensures that software is:

- a) Written faster
- b) Always deployable ✔
- c) Skipped during testing
- d) Only manually released

VU Facts

Long Questions (Practical / Lab-Based)

Q1. Explain DevOps and its role in modern software development.

Answer:
Introduction: DevOps is a set of practices that integrates software development (Dev) and IT operations (Ops) to improve collaboration and productivity.

Explanation:

- Bridges the gap between development and operations teams
- Automates build, test, and deployment processes
- Reduces deployment failures and errors
- Enables faster software delivery
- Supports continuous feedback and improvement

Example: Jenkins or GitLab CI/CD pipelines are commonly used to implement DevOps in real projects.

Conclusion: DevOps ensures reliable software delivery with higher quality and efficiency.

Q2. Explain Continuous Integration (CI) and Continuous Delivery (CD).

Answer:
Introduction: CI/CD are key DevOps practices for automating software integration and deployment.

Continuous Integration (CI):

- Developers frequently merge code into a shared repository
- Automated builds and tests run to detect errors early
- Ensures software remains in a deployable state

Continuous Delivery (CD):

- Automates the software release process
- Software is always ready for deployment
- Reduces manual intervention and deployment risks

Tools: Jenkins, Travis CI, GitLab CI/CD

Conclusion: CI/CD accelerates software delivery while maintaining quality and reliability.

VU Facts

Q3. Describe essential DevOps tools used in software construction.

Answer:
Introduction: Several tools support DevOps practices in software construction.

Main Tools:

- **Git** – Version control for managing code changes
- **Jenkins** – CI automation
- **Docker** – Containerization for environment consistency
- **Kubernetes** – Container orchestration
- **Ansible, Chef, Puppet** – Configuration management
- **Terraform, CloudFormation** – Infrastructure automation
- **Prometheus, Grafana, ELK Stack** – Monitoring and logging

Conclusion: These tools streamline development, deployment, and monitoring for efficient software construction.

Long Questions (Conceptual)

Q4. Why is Operating System and Networking knowledge important in DevOps?

Answer:
Introduction: Understanding operating systems and networking is critical in DevOps for managing servers and networked services.

Explanation:

- Operating systems manage server resources and environment configuration
- Networking ensures proper communication between services
- System administration ensures security and reliability
- Virtualization improves resource utilization and scalability
- CI/CD pipelines depend on stable OS and network configurations

Conclusion: OS and networking skills are essential for reliable, efficient, and secure DevOps operations.

☑ WEEK 12 — HETEROGENEOUS SYSTEMS

VU Facts

MCQs (Lab / Fact-Based)

1. A heterogeneous system consists of:

- a) Same type of processors
- b) Different types of components ✔
- c) Only CPUs
- d) Only GPUs

2. Which component is best for parallel processing?

- a) CPU
- b) DSP
- c) GPU ✔
- d) FPGA

3. Task partitioning in heterogeneous systems means:

- a) Combining tasks
- b) Dividing tasks among components ✔
- c) Removing tasks
- d) Storing tasks

4. Which is a programming challenge in heterogeneous systems?

- a) Uniform architecture
- b) Easy programming
- c) Diverse programming models ✔
- d) Low performance

5. Heterogeneous systems improve:

- a) Only cost
- b) Only storage
- c) Performance and energy efficiency ✔
- d) Network speed

MCQs (Conceptual)

1. The main purpose of heterogeneous systems is to:

- a) Simplify systems

VU Facts

- b) Optimize performance ✔
- c) Reduce components
- d) Avoid parallelism

2. GPUs are preferred because they:

- a) Handle sequential tasks
- b) Support parallel processing ✔
- c) Replace CPUs
- d) Reduce memory

3. Energy efficiency is achieved by:

- a) Using one processor
- b) Using specialized components ✔
- c) Increasing power
- d) Ignoring workloads

4. Task partitioning ensures:

- a) Workload on single component
- b) Efficient use of system resources ✔
- c) Removing tasks
- d) Combining tasks

5. Programming heterogeneous systems is complex due to:

- a) Same architecture
- b) Single programming model
- c) Diverse components and architectures ✔
- d) Low performance

Long Questions (Practical / Lab-Based)

Q1. What are heterogeneous systems? Give examples.

Answer:
Introduction: Heterogeneous systems use different types of hardware components to perform tasks efficiently.

Explanation:

- Combine CPUs, GPUs, DSPs, FPGAs, or specialized accelerators

VU Facts

- Each component is optimized for specific tasks
- Improves performance and efficiency for complex workloads

Examples:

- CPU + GPU system for parallel and sequential tasks
- CPU + FPGA or CPU + DSP systems for specialized computation

Conclusion: Heterogeneous systems provide higher performance and better resource utilization.

Q2. Explain task partitioning in heterogeneous systems.

Answer:
Introduction: Task partitioning divides workloads among system components according to their strengths.

Explanation:

- CPU handles control and sequential tasks
- GPU executes parallel processing tasks
- Specialized accelerators handle specific computations
- Proper partitioning ensures efficient resource utilization and high performance

Conclusion: Task partitioning is essential for maximizing heterogeneous system efficiency.

Long Questions (Conceptual)

Q3. Discuss programming challenges in heterogeneous systems.

Answer:
Introduction: Programming heterogeneous systems is complex due to different architectures and programming models.

Explanation:

- Different hardware components require specialized APIs
- Multiple programming models complicate coordination
- Synchronization and data transfer management are critical
- Memory management varies between components

Conclusion: Efficient programming requires careful planning to utilize all components effectively.

VU Facts

Q4. Why are heterogeneous systems considered energy efficient?

Answer:
Introduction: Heterogeneous systems optimize energy use by assigning tasks to components best suited for them.

Explanation:

- Tasks are allocated to components with appropriate power profiles
- Reduces unnecessary processing
- Improves performance-per-watt
- Specialized accelerators handle specific workloads efficiently

Conclusion: Energy-efficient task allocation makes heterogeneous systems suitable for high-performance applications.

☑ WEEK 13 — GITLAB

MCQs (Lab / Fact-Based)

1. Which command initializes a new Git repository?

- a) git clone
- b) git init ✔
- c) git push
- d) git commit

2. Which command is used to save changes in Git?

- a) git init
- b) git push
- c) git commit ✔
- d) git status

3. Which command uploads local commits to the remote repository?

- a) git pull
- b) git push ✔
- c) git clone
- d) git commit

VU Facts

4. In GitLab, creating a new project is necessary to:

- a) Initialize version control ✔
- b) Execute CI/CD
- c) Merge branches automatically
- d) Install Git

MCQs (Conceptual)

1. Git is mainly used for:

- a) Version control ✔
- b) Containerization
- c) Automated testing
- d) Compilation

2. The purpose of git commit is to:

- a) Upload files to remote
- b) Record snapshot of changes ✔
- c) Initialize repository
- d) Merge branches

3. Git push is important because:

- a) It deletes files locally
- b) Uploads commits to the remote repository ✔
- c) Creates a new branch
- d) Runs unit tests

Long Questions (Practical / Lab-Based)

Q1. Explain the process of creating a new project and pushing code in GitLab.

Answer:
Introduction: GitLab is a platform that allows version control and collaborative software development. Creating a new project provides a remote repository for managing code.

Explanation:

- **Step 1:** Create a new project in GitLab using the "New Project" option.

VU Facts

- **Step 2:** Initialize a local repository using `git init` on your system.
- **Step 3:** Make changes in your files and stage them using `git add`.
- **Step 4:** Commit your changes with `git commit -m "message"` to save snapshots.
- **Step 5:** Push the committed changes to GitLab using `git push`.

Conclusion: This process ensures that code changes are safely stored and shared with team members for collaboration.

✅ WEEK 14 — UNIT TESTING

MCQs (Lab / Fact-Based)

1. Unit testing focuses on testing:

- a) Entire system
- b) Individual units or components ✔
- c) User interface only
- d) Network connections

2. Isolation in unit testing is achieved by:

- a) Using breakpoints
- b) Using mock objects or stubs ✔
- c) Running system tests
- d) Merging all modules

3. Which Java framework is commonly used for unit testing?

- a) JUnit ✔
- b) Selenium
- c) Jenkins
- d) Docker

4. What does "Run with coverage" do in JUnit?

- a) Deletes untested code
- b) Executes tests and collects code coverage ✔
- c) Merges branches
- d) Compiles code

VU Facts

MCQs (Conceptual)

1. The main purpose of unit testing is to:

- a) Test user interface
- b) Verify each unit performs as expected ✔
- c) Test network speed
- d) Deploy application

2. High code coverage in unit testing indicates:

- a) All lines of code are error-free
- b) Most code has been exercised by tests ✔
- c) The software is fully secure
- d) Only main modules are tested

3. Assert methods in Java are used for:

- a) Debugging only
- b) Verifying expected outcomes ✔
- c) Version control
- d) Deployment

Long Questions (Practical / Lab-Based)

Q1. Explain unit testing and its importance in software development.

Answer:
Introduction: Unit testing is a software testing methodology that tests individual units or components of an application in isolation.

Explanation:

- Each function or method is tested independently using testing frameworks like JUnit.
- Isolation is achieved using mock objects or stubs to simulate dependencies.
- Automated unit tests allow frequent execution and support continuous integration.
- Early detection of defects reduces development costs and improves reliability.

Example:

```
@Test
void twoPlusTwoShouldEqualFour() {
  var calculator = new SimpleCalculator();
```

VU Facts

```
  assertEquals(4, calculator.add(2,2));
}
```

Conclusion: Unit testing ensures code correctness, maintainability, and reliable software delivery.

Q2. Explain "Run with coverage" and assert methods in JUnit.

Answer:
Introduction: "Run with coverage" helps track which parts of the code are exercised by tests, while assert methods verify expected outcomes.

Explanation:

- Running with coverage uses tools like JaCoCo to measure lines, branches, and statements covered.
- Assert methods include:
  - `assertEquals` – checks equality
  - `assertNotEquals` – checks inequality
  - `assertTrue` / `assertFalse` – checks boolean conditions
  - `assertNull` / `assertNotNull` – checks object references
  - `assertThrows` – verifies expected exceptions

Example:

```
assertThrows(IllegalArgumentException.class, () -> {
grader.determineLetterGrade(-1); });
```

Conclusion: These techniques ensure robust and reliable unit testing of software components.

☑ WEEK 15 — BREAKPOINTS & DEBUGGING

MCQs (Lab / Fact-Based)

1. Breakpoints are used to:

- a) Pause code execution ✔
- b) Run unit tests
- c) Merge branches
- d) Upload files

VU Facts

2. Which of the following is NOT a type of breakpoint in C#?

- a) Standard Breakpoint
- b) Conditional Breakpoint
- c) Tracepoint
- d) Git Breakpoint ✔

3. Console.WriteLine outputs messages to:

- a) Output window only
- b) Console window ✔
- c) Git repository
- d) IDE logs only

4. Debug.WriteLine outputs messages:

- a) To console during normal execution
- b) To Output window during debugging ✔
- c) To GitLab
- d) To JUnit reports

MCQs (Conceptual)

1. Breakpoints are mainly used for:

- a) Compilation
- b) Debugging ✔
- c) Testing coverage
- d) Version control

2. Console.WriteLine is used for:

- a) Debug-only output
- b) Application output ✔
- c) Version control
- d) Unit testing

3. Debug.WriteLine is useful for:

- a) Logging information while debugging ✔
- b) Running CI/CD
- c) Code coverage
- d) Containerization

VU Facts

Long Questions (Practical / Lab-Based)

Q1. Explain breakpoints in C# and their types.

Answer:
Introduction: Breakpoints are used in C# for debugging, allowing developers to pause code execution to inspect program state.

Explanation:

- **Standard Breakpoint:** Pauses execution at a specific line
- **Conditional Breakpoint:** Pauses when a condition is true
- **Hit Count Breakpoint:** Pauses after a line is hit a certain number of times
- **Tracepoint:** Logs a message instead of pausing
- **Function Breakpoint:** Pauses when a specific function is called
- **Data Breakpoint:** Pauses when a variable changes
- **Exception Breakpoint:** Pauses when an exception occurs

Conclusion: Breakpoints help identify logical errors and monitor variable values during program execution.

Q2. Explain Console.WriteLine and Debug.WriteLine in C#.

Answer:
Introduction: Both Console.WriteLine and Debug.WriteLine are used to output messages, but in different contexts.

Explanation:

- **Console.WriteLine:** Displays output to the console; used in normal program execution
- **Debug.WriteLine:** Sends messages to the Output window during debugging; used for logging program flow and variable states

Conclusion: These tools help developers monitor and debug applications efficiently.

✅ WEEK 16 — AUTOMATED CODE REVIEW

---

1 MCQs (Lab / Fact-Based)

VU Facts

1. Automated code review is also known as:

- a) Dynamic code analysis
- b) Static code analysis ✔
- c) Unit testing
- d) CI/CD

2. Which of the following is checked in automated code review?

- a) Syntax and style ✔
- b) Unit test success only
- c) Version control commits
- d) Breakpoints

3. Tools like ESLint, Pylint, SonarQube are used for:

- a) Debugging
- b) Automated code review ✔
- c) CI/CD pipelines
- d) Version control

MCQs (Conceptual)

1. The main purpose of automated code review is to:

- a) Manually inspect code
- b) Automatically detect issues, vulnerabilities, and style violations ✔
- c) Run breakpoints
- d) Merge Git branches

2. Benefits of automated code review include:

- a) Consistency, efficiency, early issue detection ✔
- b) Only version control
- c) Manual code inspection
- d) Ignoring coding standards

3. Code review tools can integrate into development workflows to:

- a) Provide feedback in real-time ✔
- b) Deploy applications automatically
- c) Compile code
- d) Remove debugging messages

VU Facts

Long Questions (Practical / Lab-Based)

Q1. Explain automated code review and its importance.

Answer:
Introduction: Automated code review, also known as static code analysis, uses software tools to inspect source code for potential issues and adherence to coding standards.

Explanation:

- Checks syntax and coding style
- Detects complex or convoluted code
- Identifies code smells and security vulnerabilities
- Detects duplicated code and dependency issues
- Ensures adherence to coding standards

Tools Examples: ESLint for JavaScript, Pylint for Python, SonarQube for multiple languages, RuboCop for Ruby, Checkstyle for Java

Conclusion: Automated code review ensures consistent, secure, and maintainable code across software projects.

Q2. What are the benefits of automated code review?

Answer:
Introduction: Automated code review improves code quality and development efficiency.

Explanation:

- **Consistency:** Uniform application of coding standards
- **Efficiency:** Scans large codebases quickly
- **Early Detection:** Identifies issues before deployment
- **Security:** Detects potential vulnerabilities
- **Integration:** Provides real-time feedback in the development workflow
- **Scalability:** Handles increasing code changes easily

Conclusion: Automated code review reduces errors, ensures maintainability, and enhances overall software quality.

VU Facts